



PYTHON

Estructura y documentación

ESTRUCTURA Y DOCUMENTACIÓN

- Estructura de proyectos.
 - Estructuras
 - pip
 - Módulos y paquetes.
- Estructura del código.
 - Organización de ficheros.
 - Comentarios.
- Git.
- Documentación.

ESTRUCTURA Y DOCUMENTACIÓN

■ Estructura básica de un proyecto Python:

mi_app/

```
|
|—— app.py          # Punto de entrada
|—— requirements.txt # Dependencias
|—— README.md       # Documentación
|—— src/
|   |—— __init__.py
|   |—— models.py    # Datos / clases
|   |—— services.py  # Lógica de negocio
|   |—— utils.py     # Funciones auxiliares
|—— tests/
|   |—— __init__.py
|   |—— test_services.py
```

ESTRUCTURA Y DOCUMENTACIÓN

- Instalador de paquetes: pip
- Módulos:
 - E:\Python\Python310\Lib → Esta es la ubicación en el ordenador del profesor. ¿Dónde tienes instalado Python?
 - Copiar los módulos propios a dicha carpeta (pueden estar en carpetas para mejorar la organización)

ESTRUCTURA Y DOCUMENTACIÓN

- Comentarios:
 - La importancia de los comentarios.
 - Destinatarios de los comentarios.
- Documentación
 - Comentarios vs Documentación
- Mensajes para el usuario
- Logs (traza)

ESTRUCTURA Y DOCUMENTACIÓN

- Comentarios:

- De línea: #
- De bloque: “”””

- Tags:

- # TODO
- # FIXME
- # BUG

ESTRUCTURA Y DOCUMENTACIÓN

- Logs:
 - Módulo logging
 - Configuración:
 - Establecer nivel de filtrado de logs:
 - `logging.getLogger().setLevel(logging.level)`
 - *level:*

Nivel	Valor numérico
CRITICAL	50
ERROR	40
WARNING	30
INFO	20
DEBUG	10
NOTSET	0

ESTRUCTURA Y DOCUMENTACIÓN

- **Logs:**

- **Módulo logging**

- **Uso:**

- ```
logging.debug("Depuración")
```

- ```
logging.info("Información")
```

- ```
logging.warning("Advertencia")
```

- ```
logging.error("Error")
```

- ```
logging.critical("Error crítico")
```

- ```
logging.exception("Excepción")
```

 → Nivel ERROR. Sólo para excepciones.

ESTRUCTURA Y DOCUMENTACIÓN

- Logs:
 - Módulo logging
 - Uso Exception:

```
try:  
    b=5  
    c=0  
    a = b/c  
except ZeroDivisionError as error:  
    logging.exception(error)
```

ERROR:root:division by zero

Traceback (most recent call last):

File "h:_ProyectosPython\borrar.py", line 15, in funcion

a = b/c

ZeroDivisionError: division by zero

ESTRUCTURA Y DOCUMENTACIÓN

■ Logs:

- Módulo logging
- Configuración salida a fichero:

```
logging.basicConfig(filename="fichero_salida.log",  
                    filemode='a',  
                    format='%(asctime)s,%(msecs)d %(name)s %(levelname)s %(message)s',  
                    datefmt='%H:%M:%S',  
                    level=logging.DEBUG)
```

■ Atributos:

- <https://docs.python.org/3/library/logging.html#logrecord-attributes>

ESTRUCTURA Y DOCUMENTACIÓN

- Comentarios:

- De línea: #
- De bloque: """

- Sugerencias de tipo:

```
def dame_numero(numero : int) -> int:
    """
    Devuelve el número 10
    """
    return 10
```

Sugerencia de tipo (Type Hinting)

```
2 gc.dame_numero()  
    dame_numero(numero: int) -> int  
        numero: int  
        Devuelve el número 10
```

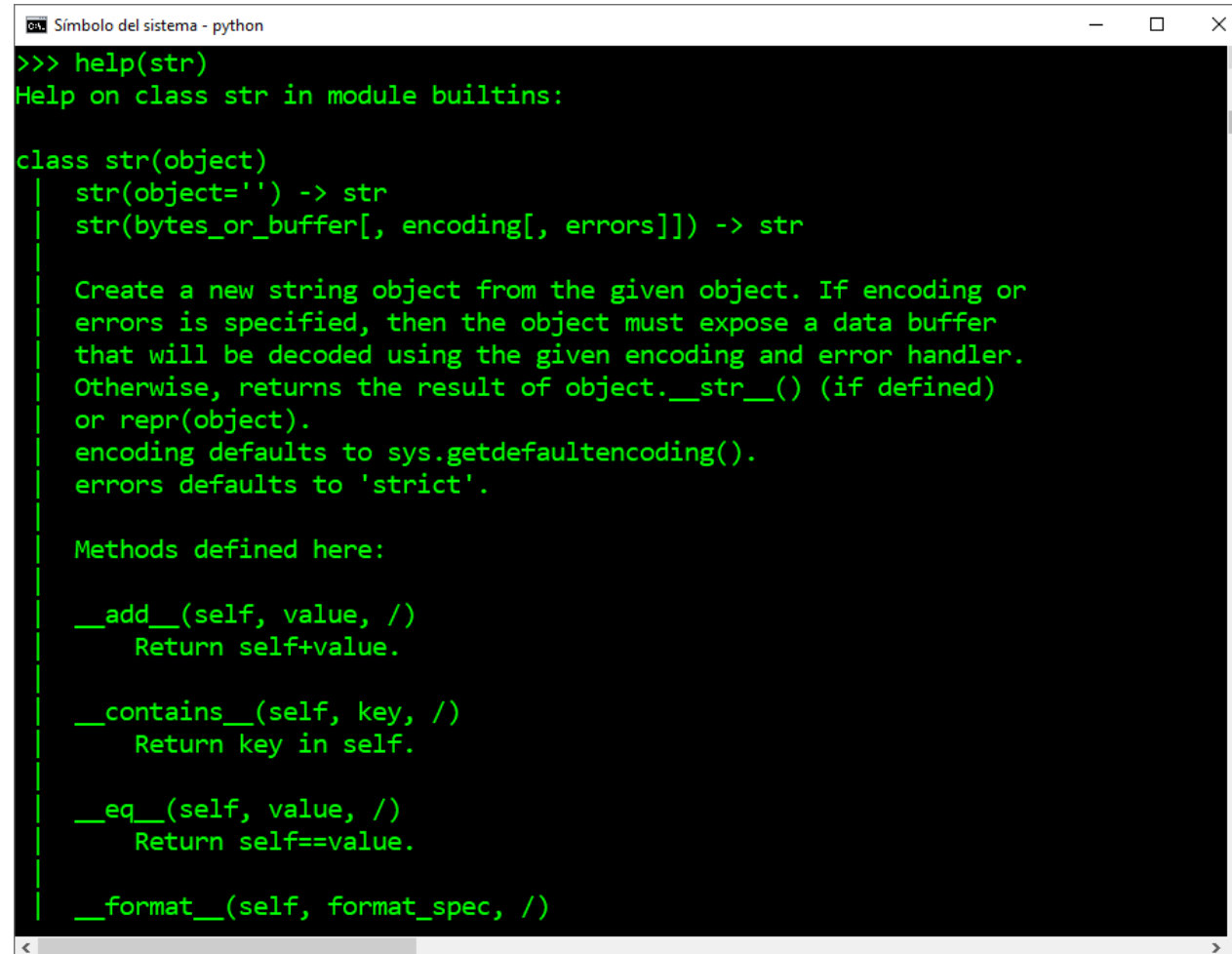
- Tags:

- # TODO
- # FIXME
- # BUG

ESTRUCTURA Y DOCUMENTACIÓN

■ Función help:

>>>help(str)

A screenshot of a Python terminal window titled 'Símbolo del sistema - python'. The terminal shows the command '>>> help(str)' and its output. The output includes the title 'Help on class str in module builtins:', the class definition 'class str(object)', the constructor signature 'str(object='') -> str' and 'str(bytes_or_buffer[, encoding[, errors]]) -> str', a detailed description of the class, and a list of methods defined for the class: __add__, __contains__, __eq__, and __format__.

```
>>> help(str)
Help on class str in module builtins:

class str(object)
|   str(object='') -> str
|   str(bytes_or_buffer[, encoding[, errors]]) -> str
|
|   Create a new string object from the given object. If encoding or
|   errors is specified, then the object must expose a data buffer
|   that will be decoded using the given encoding and error handler.
|   Otherwise, returns the result of object.__str__() (if defined)
|   or repr(object).
|   encoding defaults to sys.getdefaultencoding().
|   errors defaults to 'strict'.
|
|   Methods defined here:
|
|   __add__(self, value, /)
|       Return self+value.
|
|   __contains__(self, key, /)
|       Return key in self.
|
|   __eq__(self, value, /)
|       Return self==value.
|
|   __format__(self, format_spec, /)
```

ESTRUCTURA Y DOCUMENTACIÓN

- **Docstring en Python**

- Un *docstring* es una cadena de texto que sirve para **documentar** un módulo, función, clase o método.
- Se escribe normalmente utilizando **tres comillas** (""" ... """).
- Su objetivo es explicar **qué hace** el código y cómo utilizarlo.
- Python permite acceder a esta documentación mediante `.__doc__()`.

ESTRUCTURA Y DOCUMENTACIÓN

- **Docstring en Python**
- Los docstrings pueden aparecer en diferentes lugares
 - Módulos: explican para qué sirve un archivo Python.
 - Funciones: describen qué hace una función.
 - Clases: explican el propósito de una clase.
 - Métodos: documentan las operaciones de un objeto.

```
class Persona:  
    """Representa a una persona."""  
  
    def saludar(self):  
        """Muestra un saludo por pantalla."""  
        print("Hola")
```

ESTRUCTURA Y DOCUMENTACIÓN

- **Docstring en Python**
- **Un buen docstring debe ser claro y útil**
 - Debe explicar, cuando sea necesario:
 - Qué hace la función o clase.
 - Qué parámetros recibe.
 - Qué devuelve.
 - Qué excepciones puede producir.
 - Cualquier comportamiento importante.

ESTRUCTURA Y DOCUMENTACIÓN

- **Docstring en Python**
- **Documentar funciones más complejas**
 - Los docstrings pueden incluir información detallada sobre los parámetros y el valor devuelto.

```
def calcular_area(base, altura):  
    """  
    Calcula el área de un triángulo.  
  
    Parámetros:  
    base: Base del triángulo.  
    altura: Altura del triángulo.  
  
    Retorna:  
    El área del triángulo.  
    """  
    return (base * altura) / 2
```

ESTRUCTURA Y DOCUMENTACIÓN

■ Docstring y la función help:

```
def dame_numero(numero : int) -> int:
```

```
    """
```

```
    Devuelve el número 10
```

```
    """
```

```
    return 10
```

```
>>> help(paniagua.generador_cosas)
Help on module paniagua.generador_cosas in paniagua:

NAME
    paniagua.generador_cosas

MODULE REFERENCE
    https://docs.python.org/3.9/library/paniagua.generador_cosas

    The following documentation is automatically generated from the Python
    source files. It may be incomplete, incorrect or include features that
    are considered implementation detail and may vary between Python
    implementations. When in doubt, consult the module reference at the
    location listed above.

FUNCTIONS
    dame_numero(numero: int) -> int
        Devuelve el número 10

FILE
    e:\python\python39\lib\paniagua\generador_cosas.py
```